

Shaders

Lect. Sibsan Suksuchano

Shader

a **shader** is a programmable operation which is applied to data as it moves through the rendering pipeline.

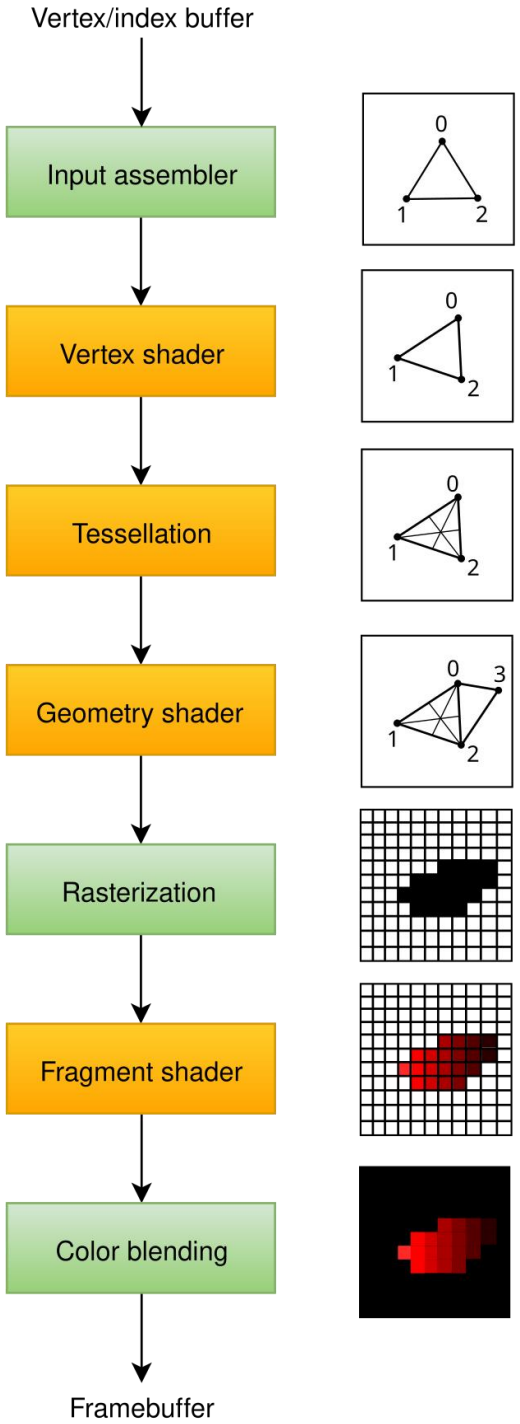
A *shader* specifies operations that are meant to occur on data as it moves through different programmable stages of the OpenGL processing pipeline, starting with vertices specified by the application and ending with fragments prior to being written to the framebuffer. The programming language used for shaders is described in the OpenGL Shading Language Specification.

Copyright © 2006-2022 The Khronos Group Inc. All Rights Reserved.

Shader

- Vertex Shader

- Fragment Shader
  (Pixel Shader)



4.0

Vertices ✪

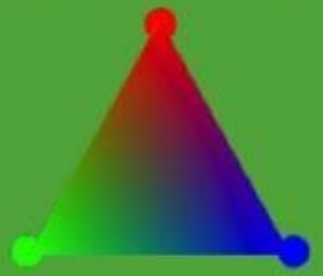


VERTEX SHADER

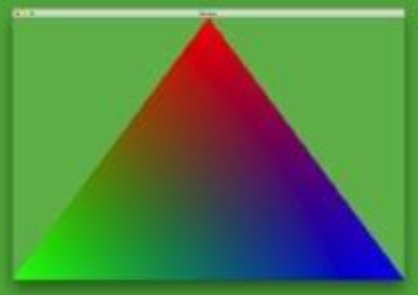
RASTERIZER

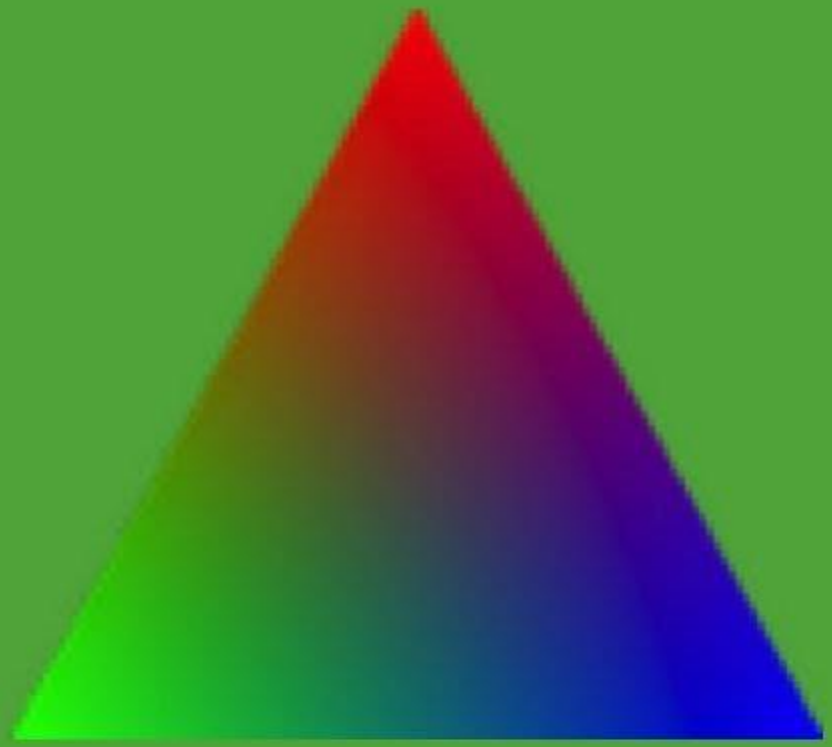


FRAGMENT SHADER

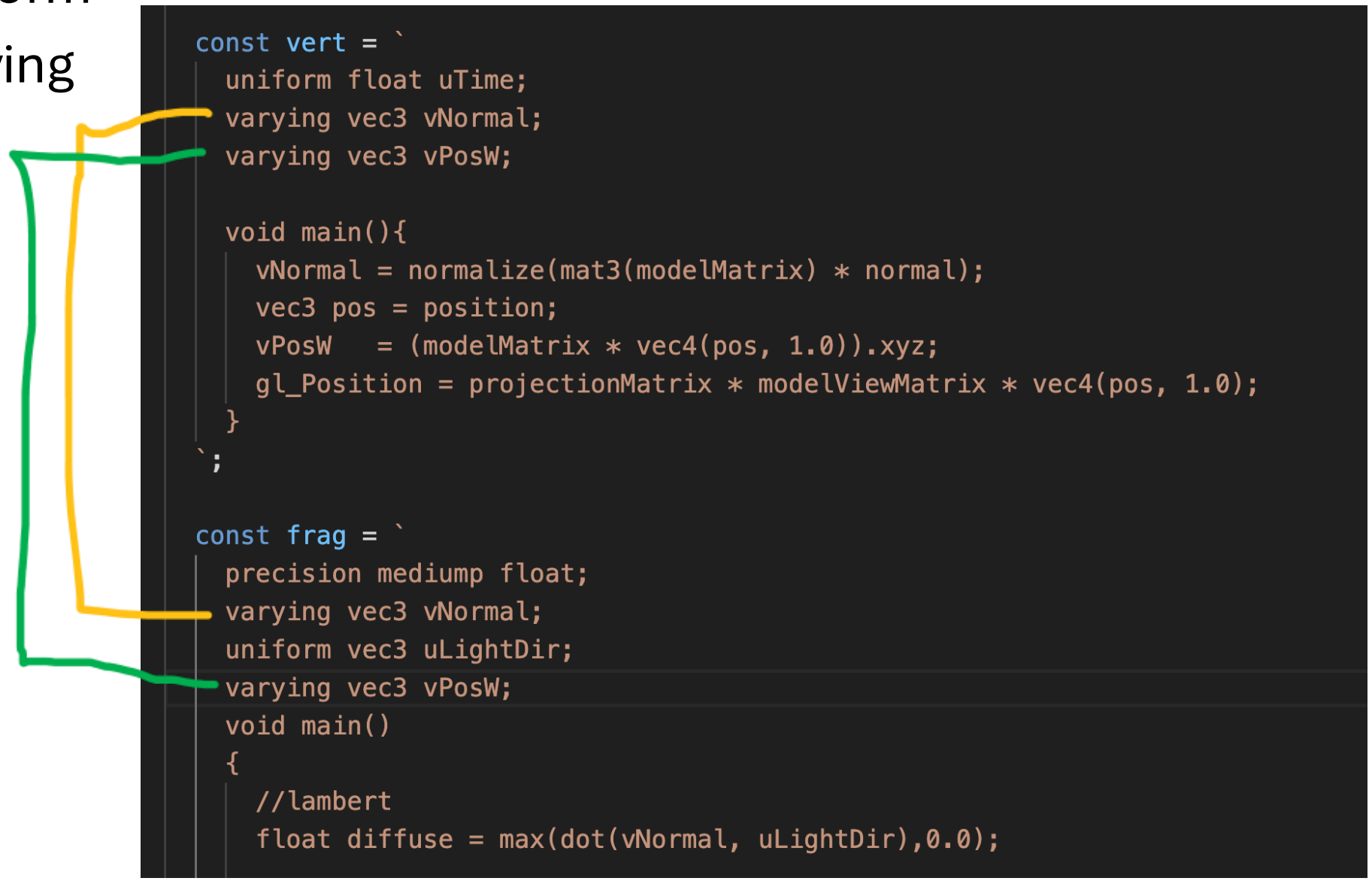


RETURN PIXELS





- uniform
- varying



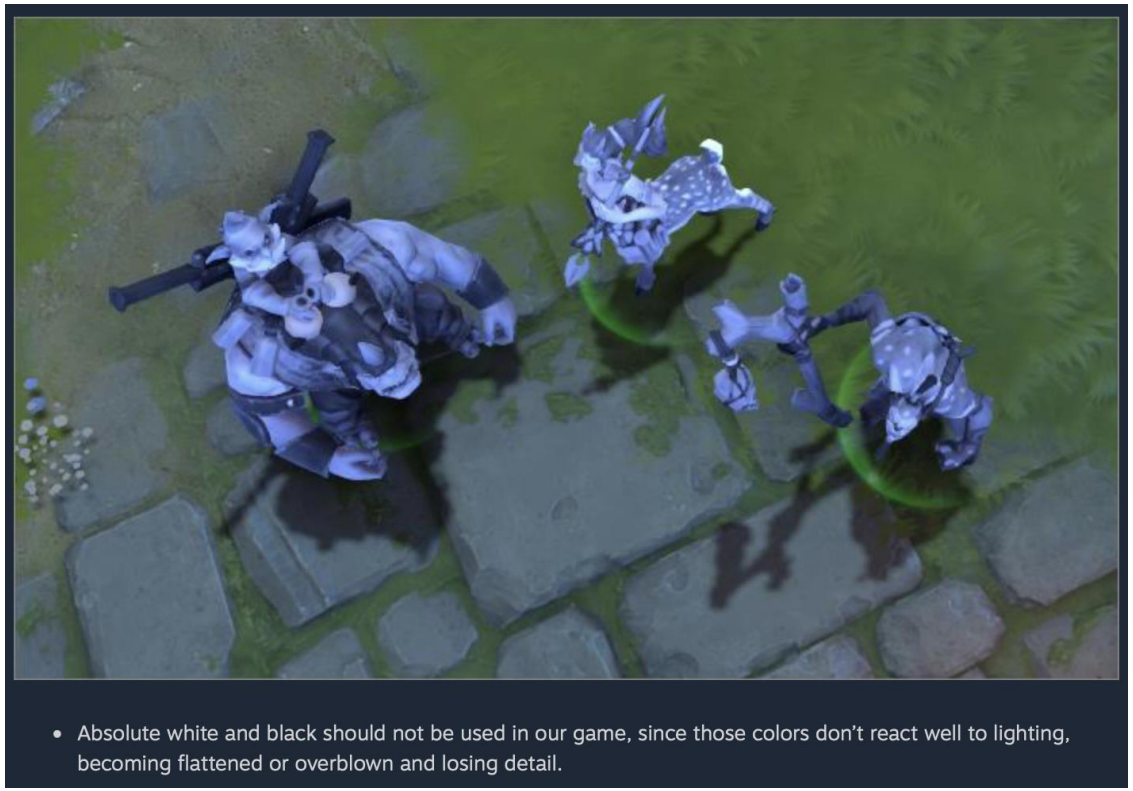
```
const vert = `
  uniform float uTime;
  varying vec3 vNormal;
  varying vec3 vPosW;

  void main(){
    vNormal = normalize(mat3(modelMatrix) * normal);
    vec3 pos = position;
    vPosW   = (modelMatrix * vec4(pos, 1.0)).xyz;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
  }
`;

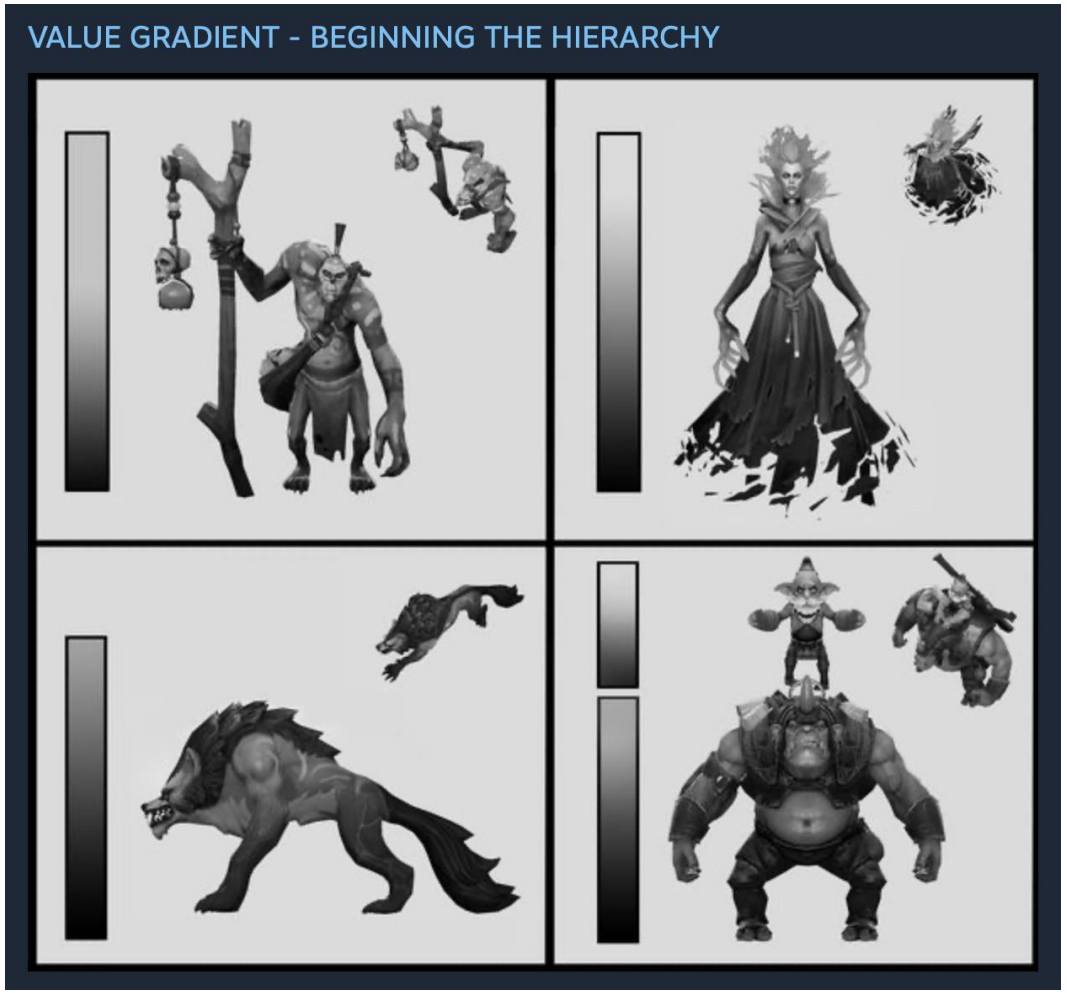
const frag = `
  precision mediump float;
  varying vec3 vNormal;
  uniform vec3 uLightDir;
  varying vec3 vPosW;
  void main()
  {
    //lambert
    float diffuse = max(dot(vNormal, uLightDir),0.0);
```

Basic Shader

https://help.steampowered.com/en/faqs/view/0688-7692-4D5A-1935



- Absolute white and black should not be used in our game, since those colors don't react well to lighting, becoming flattened or overblown and losing detail.





https://www.youtube.com/watch?v=WR_SP4LmOlw

Vertex Shader

Shader สำหรับทำงานกับ vertex เช่น

- เปลี่ยนแปลง/ใช้งาน ตำแหน่ง vertex (position)
- เปลี่ยนแปลง/ใช้งานทิศทางของ vertex (normal)
- เปลี่ยนแปลง/ใช้งาน uv ของ vertex (uv)
- เปลี่ยนแปลง/ใช้งานค่าสี vertex (color)

Attribute สำคัญใน Vertex Shader

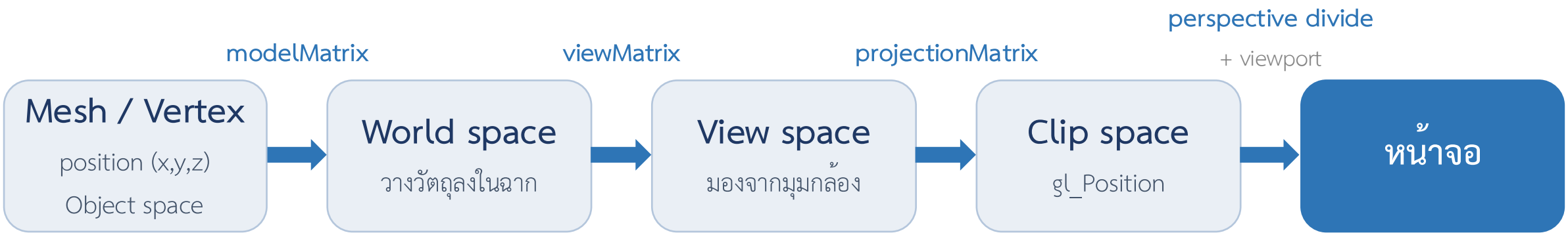
Attribute	Data Type	Description
position	vec3	ตำแหน่งแต่ละ vertex ใน object/local space
normal	vec3	Normal vector ของ vertex ใน local
uv	vec2	พิกัด ของ texture บน object
color	vec3 | vec4	สีของ vertex (Vertex Color) (ถ้ากำหนด)
tangent	vec4	Tangent สำหรับ normal mapping
skinIndex	vec4	ลำดับกระดูก (สำหรับ animation ใช้กระดูกควบคุม vertices)
skinWeight	vec4	น้ำหนักที่กระดูกส่งผลต่อ vertices

Matrix สำคัญใน Vertex Shader

```
gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
```

Matrix	Data Type	Description
modelMatrix	mat4	Transform จาก Object space เป็น World space
viewMatrix	mat4	Transform จาก World space เป็น View space
projectionMatrix	mat4	Transform จาก View space เป็น Clip space
modelViewMatrix	mat4	viewMatrix * modelMatrix (Obj space เป็น View) (ใช้บ่อยจึงคูณไว้ให้เลย)
normalMatrix	mat3	Transform จาก normal เป็น View space
cameraPosition	vec3	พิกัดกล้อง

จากข้อมูล mesh สู่ตำแหน่งบนจอ



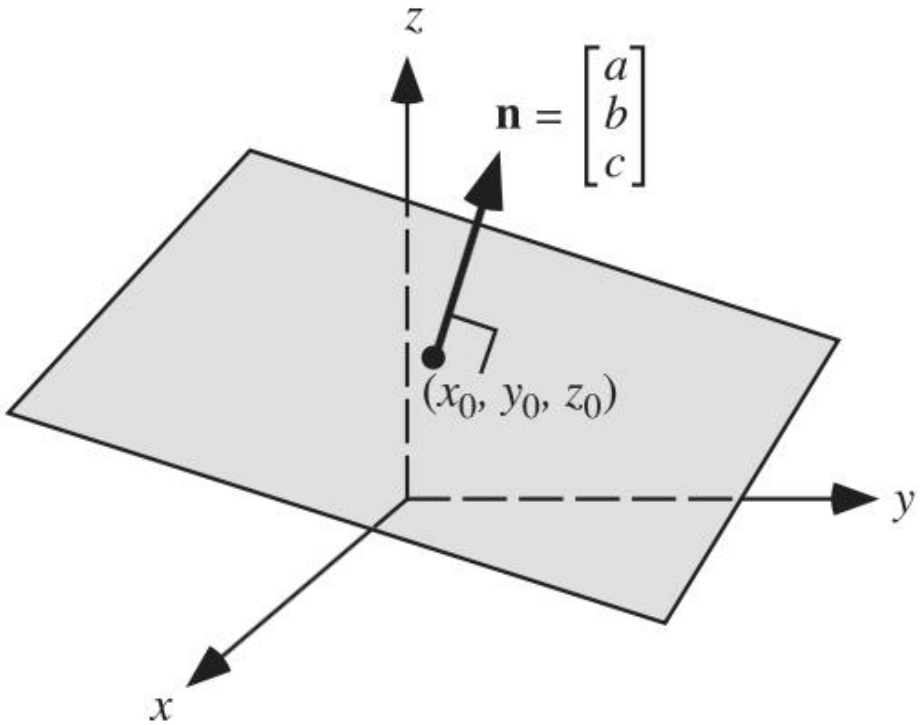

```
gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.0)
```

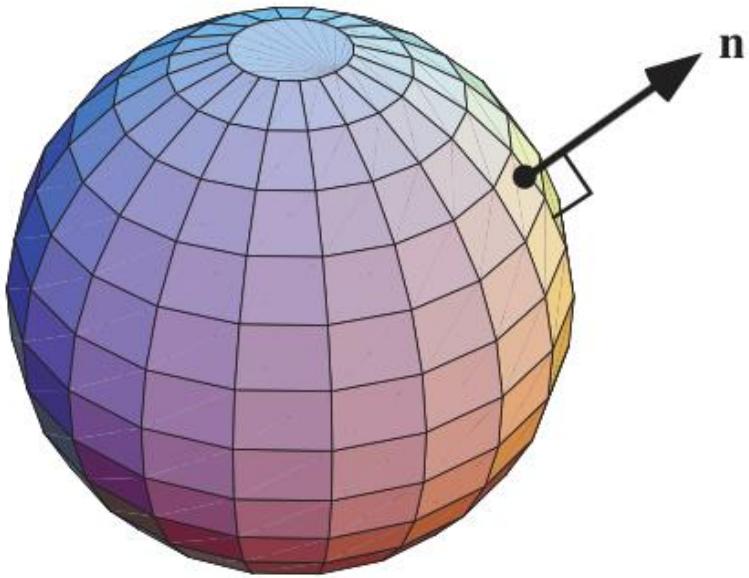
Vertex Shader รับพิกัด vertex ของ mesh แล้วคูณเมทริกซ์ทีละขั้น จนได้ตำแหน่งบนจอ

Vertex Shader : position

```
const vert = `
void main(){
	float PI = 3.141592653589793;
	vec3 pos = position;
	pos.z = sin(pos.x * PI);
	gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
`;
```

Vertex Shader : normal





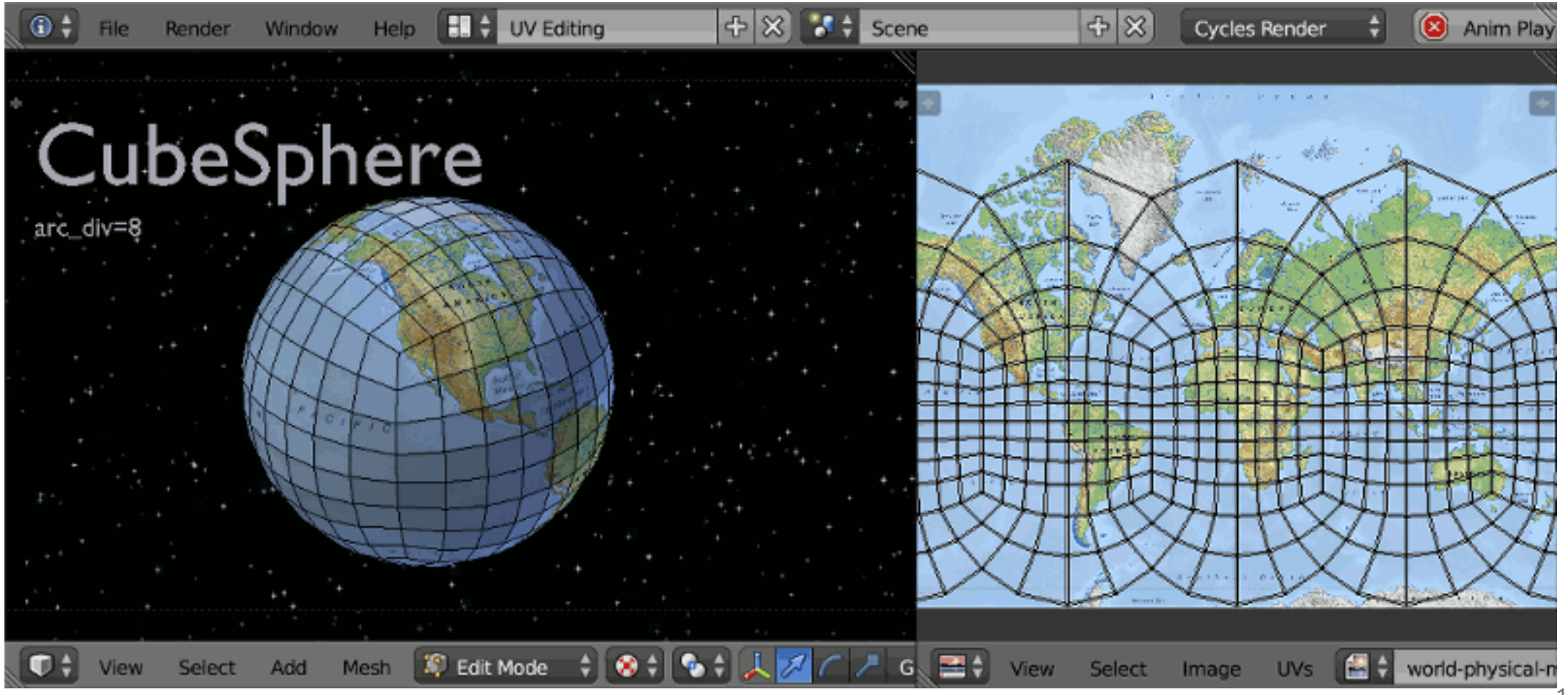
Vertex Shader : normal



```
const vert = `
uniform float uTime;
void main(){
	float PI = 3.141592653589793;
	vec3 pos = position;
	//pos.z = sin(pos.x * PI);
	vec3 N = normalize(normal);
	pos += N * sin(pos.x * PI + uTime) * 0.2; // ปรับความสูงได้ด้วยการคูณ factor


	gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
`;
```

Vertex Shader : uv



Vertex Shader : uv

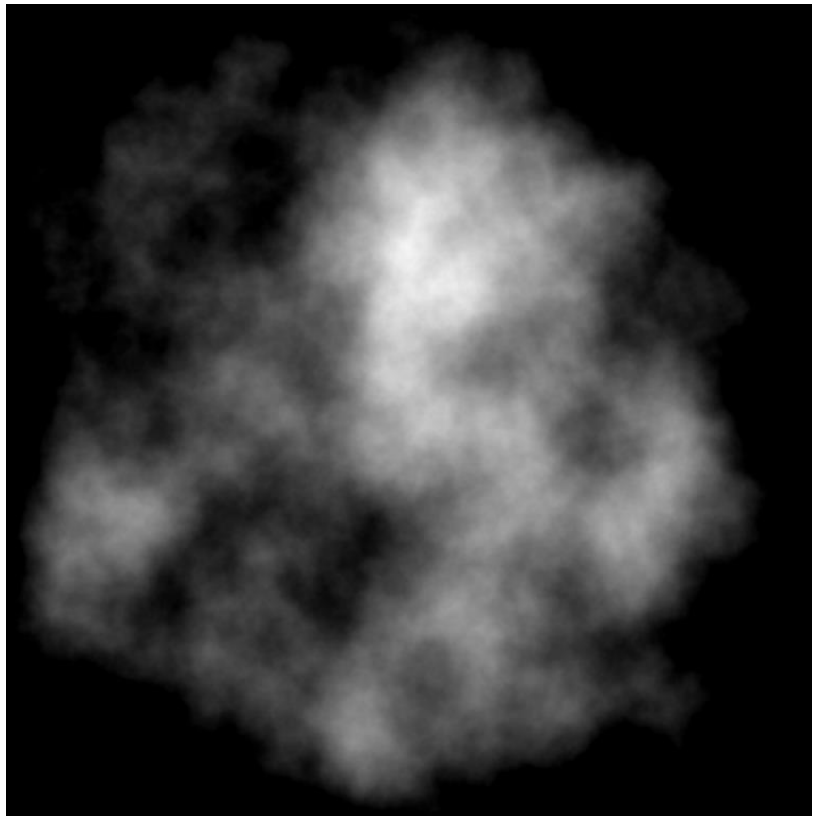
```
const vertexShader = `
uniform float uTime;
varying vec2 vUv;

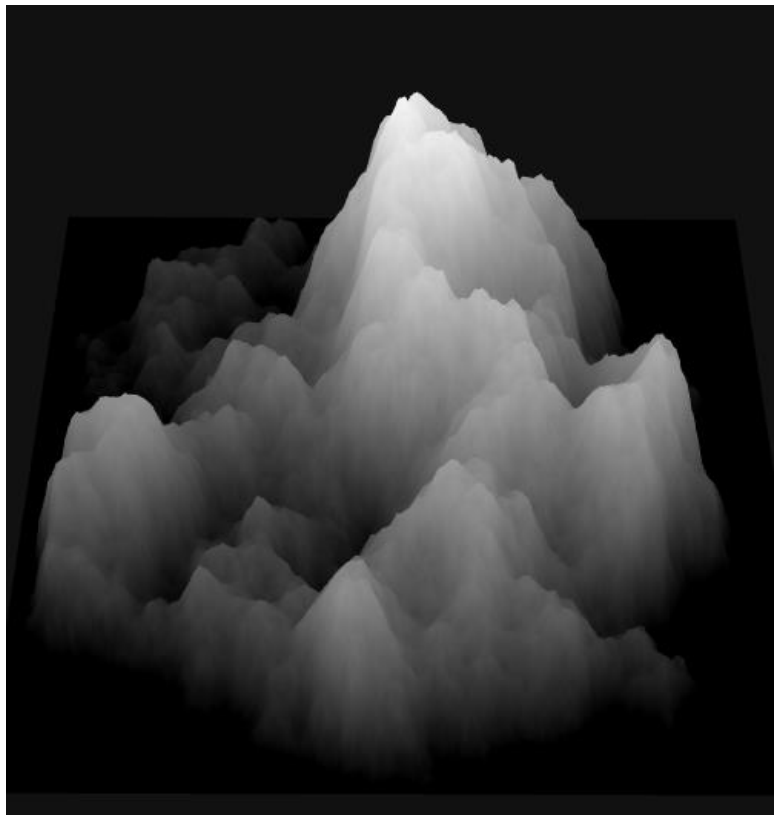

void main() {
vUv = uv;


float waveX = sin(uv.x * 2.0 + uTime * 2.0);
float waveY = sin(uv.y * 2.0 + uTime * 2.0);
float displacement = waveX * waveY;
vec3 pos = position + normal * displacement;
gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
`;
```

```
const uniforms = {
uTime: { value: 0.0 },
uTexture: { value: new THREE.TextureLoader().load('https://bwiggs.com/projects/terrain/images/heightmap-final.png') }
};
```

```
uniform sampler2D uTexture;
//ตัวอย่าง uv
vHigh = texture2D(uTexture, uv).r;
pos.z = pos.z + vHigh;
```





```
const vertexShader = `
        uniform float uTime;
        uniform sampler2D uTexture;
        varying vec2 vUv;
        varying float vHigh;
        void main() {
                vUv = uv;
                vec3 pos = position;
                // เริ่มเขียน vertex shader ที่นี่

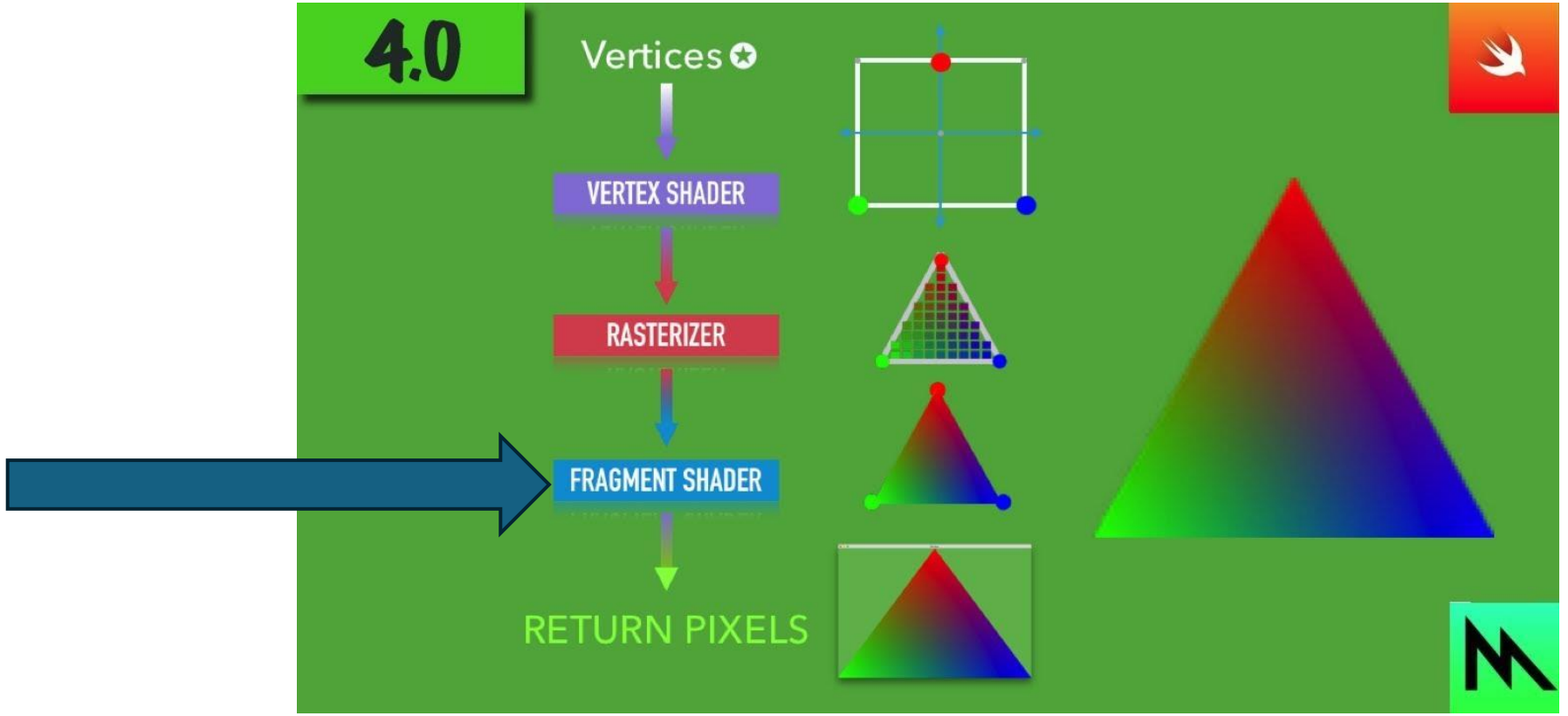

                //ตัวอย่าง position
                //pos.y = pos.y + sin(pos.x * 4.0 + uTime * 8.0) * 0.1;
                //pos.z = pos.z + sin(pos.x * 1.0 + uTime * 4.0) * 0.1;
                //pos.x = pos.x + sin(pos.y * 2.0 + uTime * 4.0) * 0.2;


                //ตัวอย่าง normal
                //pos = pos + (normal * (sin(uTime)+1.0) * 0.2);

                //ตัวอย่าง uv
                vHigh = texture2D(uTexture, uv).r;
                pos.z = pos.z + vHigh;
                gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
        }
`;
```

Fragment Shader

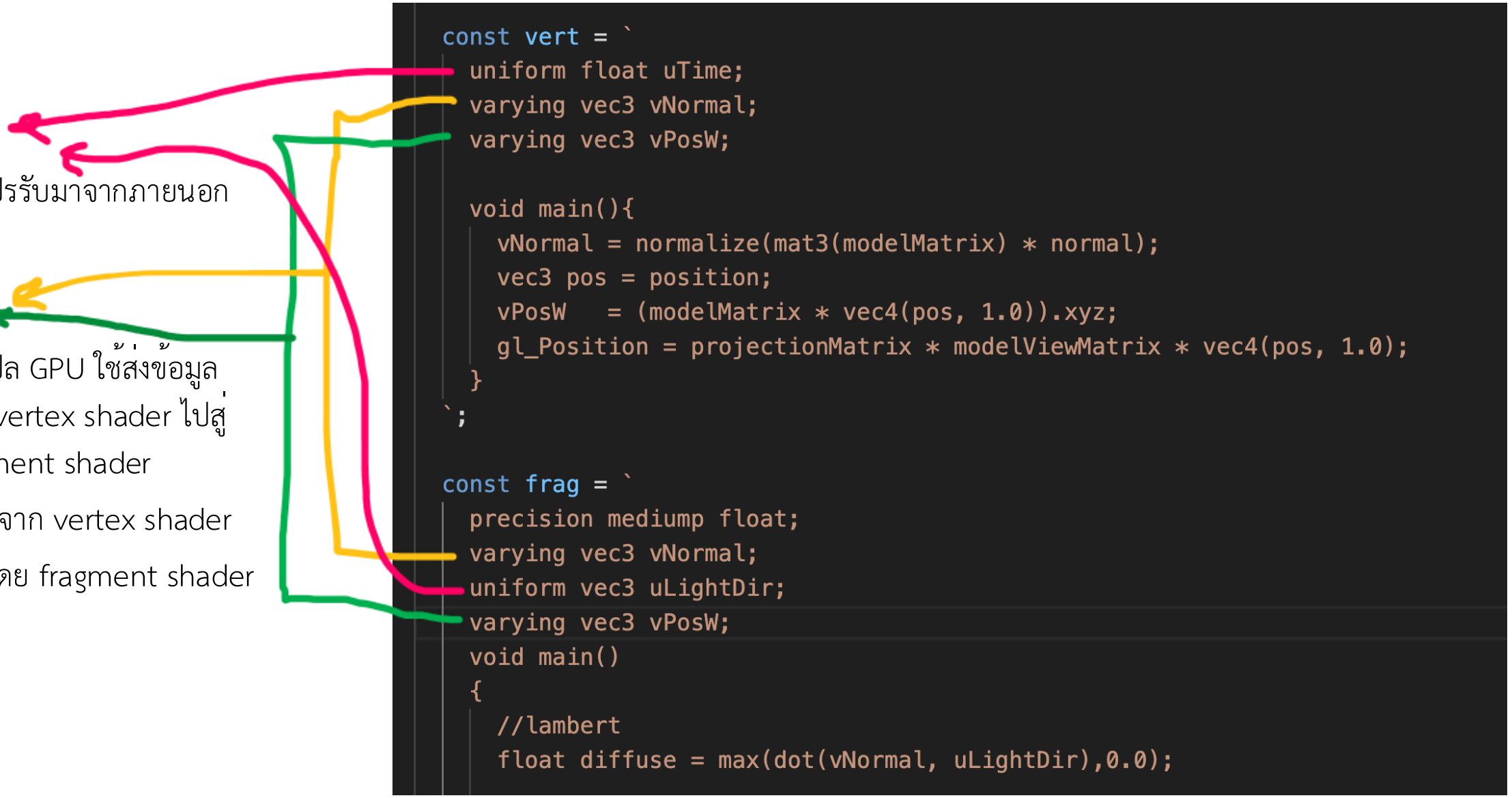
- Programming language สำหรับปรับเปลี่ยนค่าของผล rasterization





https://www.youtube.com/watch?v=8_ZTvG1WQxM

- Uniform
  - ตัวแปรรับมาจากภายนอก GPU
- Varying
  - ตัวแปล GPU ใช้ส่งข้อมูลจาก vertex shader ไปสู่ fragment shader
  - เขียนจาก vertex shader
  - อ่านโดย fragment shader



```
const vert = `
  uniform float uTime;
  varying vec3 vNormal;
  varying vec3 vPosW;

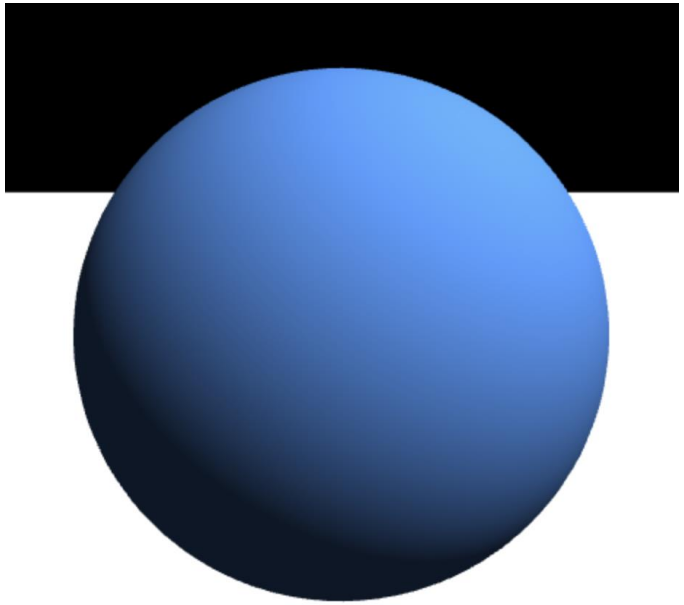
  void main(){
    vNormal = normalize(mat3(modelMatrix) * normal);
    vec3 pos = position;
    vPosW   = (modelMatrix * vec4(pos, 1.0)).xyz;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
  }
`;

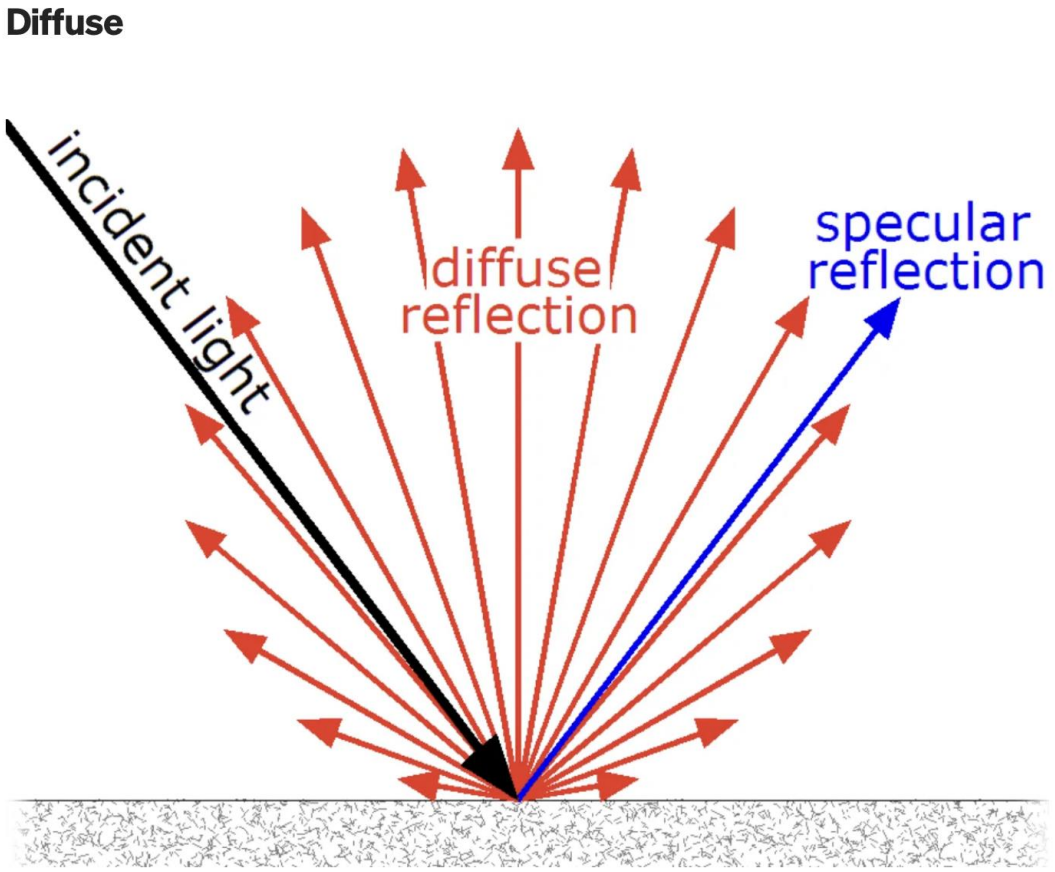
const frag = `
  precision mediump float;
  varying vec3 vNormal;
  uniform vec3 uLightDir;
  varying vec3 vPosW;
  void main()
  {
    //lambert
    float diffuse = max(dot(vNormal, uLightDir),0.0);
```

Function ที่ใช้งานบ่อย

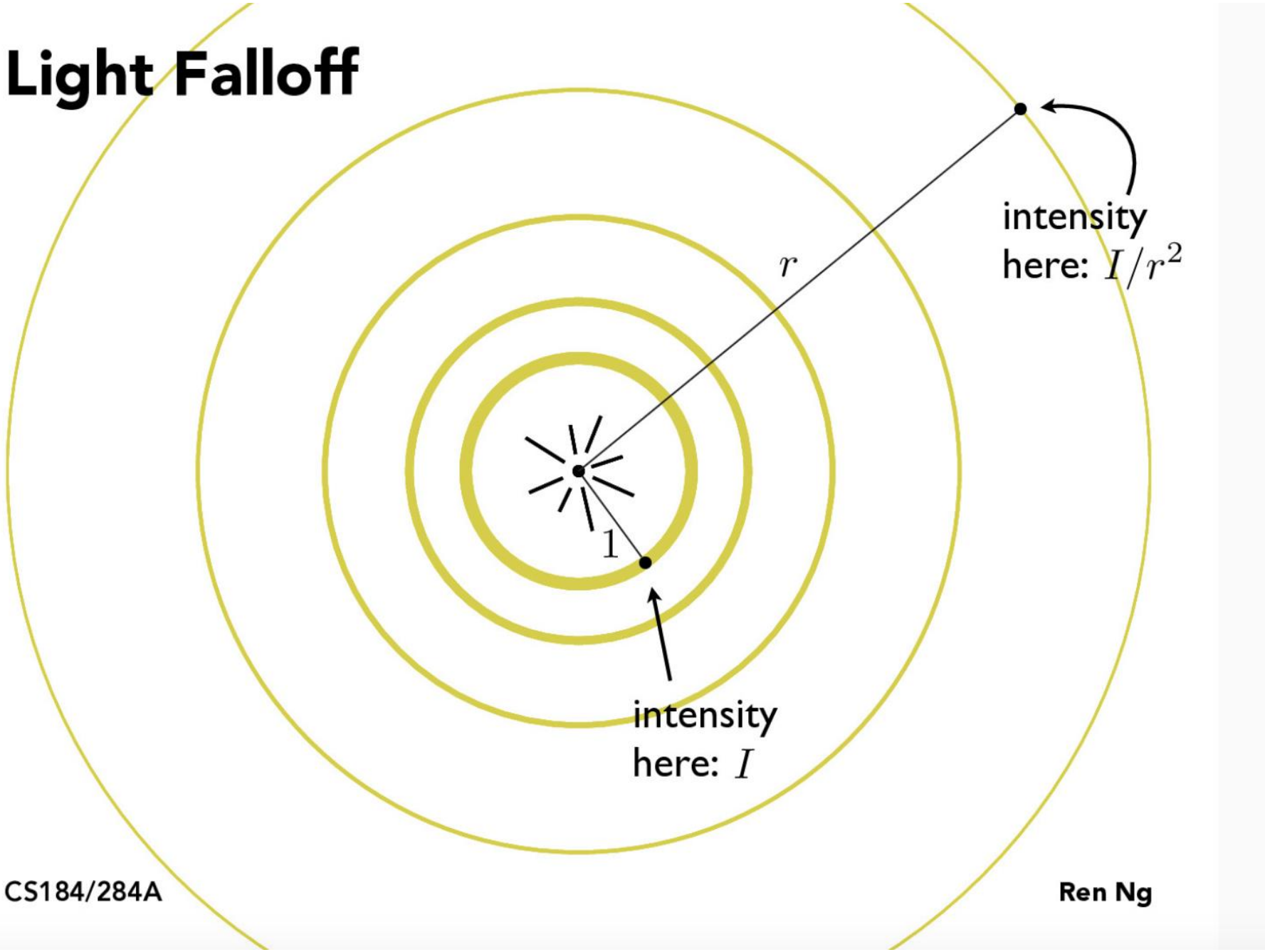
ฟังก์ชัน	ความหมายย่อ	ตัวอย่าง GLSL	ใช้บ่อยกับ
max(a,b)	เลือกค่าสูงสุด	float d = max(dot(N,L), 0.0);	diffuse
min(a,b)	เลือกค่าต่ำสุด	float m = min(a, b);	ทั่วไป
clamp(x,lo,hi)	บีบค่าให้อยู่ในช่วง	float s = clamp(v, 0.0, 1.0);	saturate
pow(x,y)	ยกกำลัง	float spec = pow(max(dot(R,V),0.0), 32.0);	specular
abs(x)	ค่าสัมบูรณ์	float a = abs(sin(t));	ทั่วไป
sign(x)	คืนเครื่องหมาย	float sg = sign(x);	masking
floor(x)	ปัดลง	float f = floor(uv.x*5.0);	grid
ceil(x)	ปัดขึ้น	float c = ceil(uv.x*5.0);	grid
fract(x)	ส่วนเศษ	float g = fract(uv.x*5.0);	tiling
mod(x,y)	เลขเศษจากหาร	vec2 uv2 = mod(uv*4.0, 1.0);	Tiling
step(t,value)	แบ่งค่าเป็น 0, 1	specular = step(0.5, specular);	
smoothstep(tl,th,value)	แบ่งค่า เป็น 0.1 โดยมี smooth	specular = smoothstep(0.4, 0.6, specular);	

Lambert Shader





Light Falloff



CS184/284A

Ren Ng

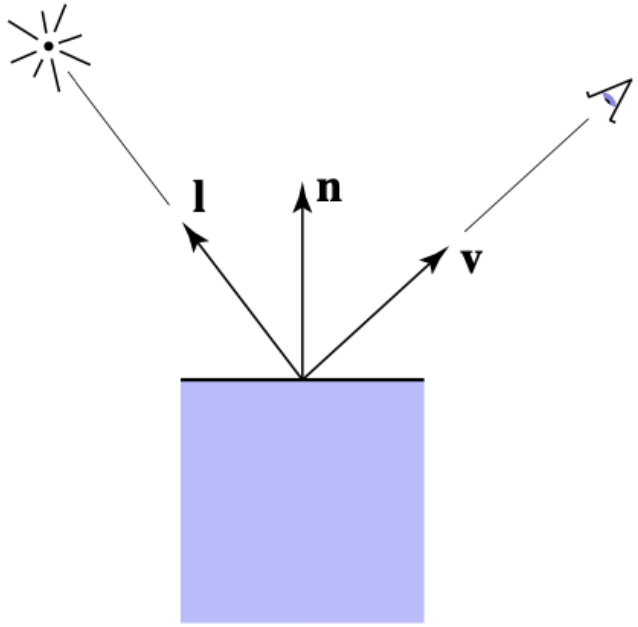
Lambert Shader

$$I = I_p k_d |N||V_L| cos(\theta)$$

Ip = ความเข้มของแสงจาก light source
k_d = ผิวสะท้อนแสงแบบ diffuse ได้มากแค่ไหน (มักจะ $0 - 1$)
N = surface normal
L = ทิศทางไปหาแสง
θ = มุมระหว่าง N กับ L

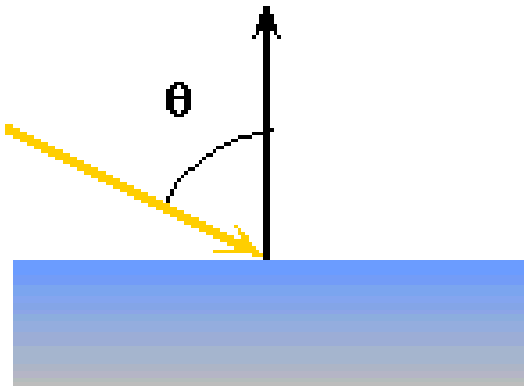
$$I_{\text{diffuse}} = k_d I_{\text{light}} \max(0, \vec{N} \cdot \vec{L})$$

- Compute light reflected toward camera
- Inputs:
  - eye direction
  - light direction (for each of many lights)
  - surface normal
  - surface parameters (color, shininess, …)



Lambert’s cosine law

$$I_{diffuse} = k_d I_{light} \cos\theta$$

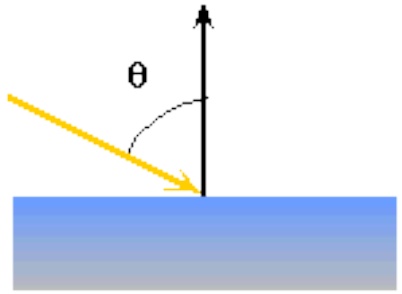


=COS(B1*3.141/180)

A	B	C	D
COS(....)	0	1.00	1.00
	15	0.97	0.97
	30	0.87	0.87
	45	0.71	0.71
	60	0.50	0.50
	75	0.26	0.26
	90	0.00	0.00

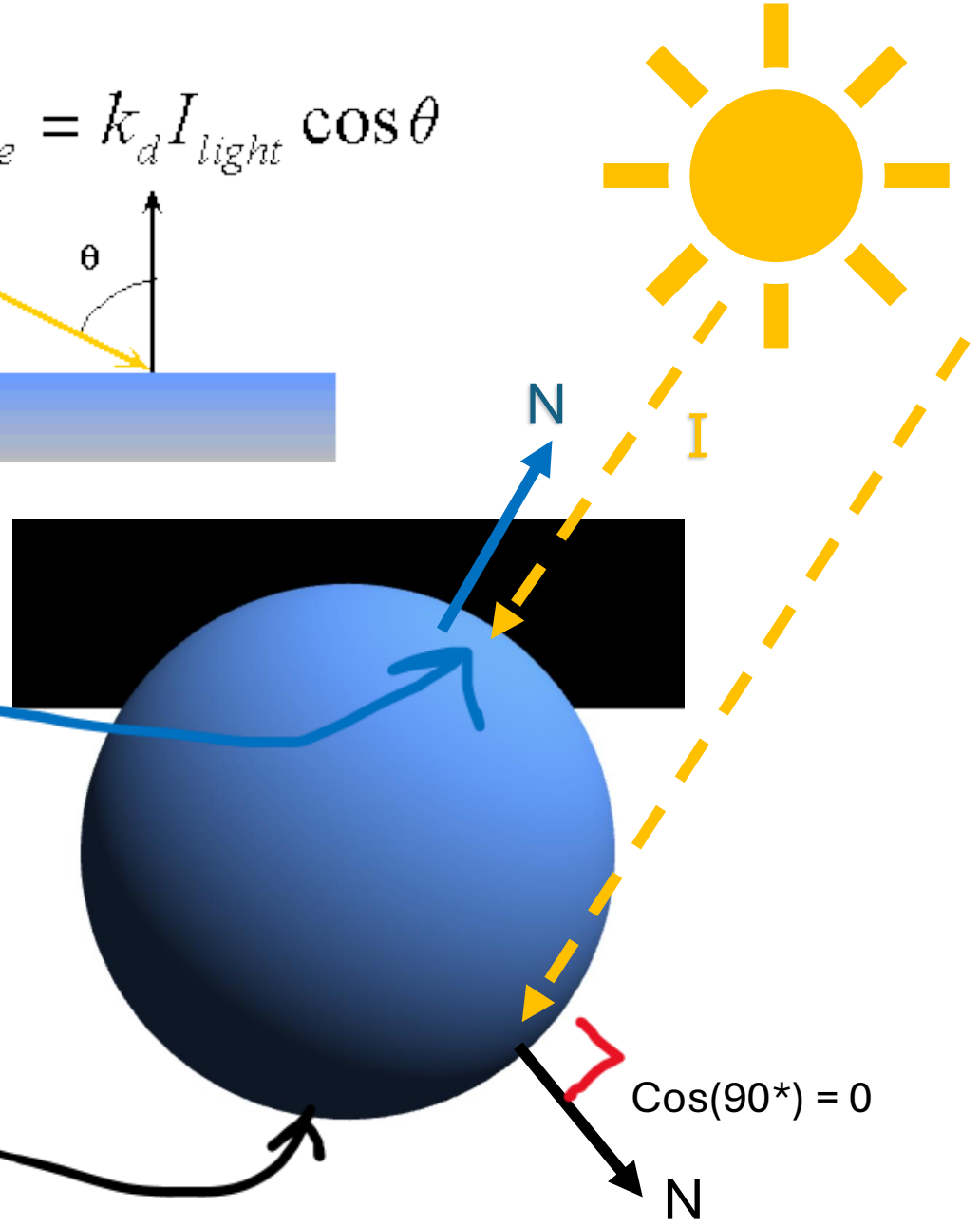
Lambert's cosine law

$$I_{diffuse} = k_d I_{light} \cos\theta$$



=COS(B1*3.141/180)

A	B	C	D
COS(....)	0	1.00	1.00
	15	0.97	0.97
	30	0.87	0.87
	45	0.71	0.71
	60	0.50	0.50
	75	0.26	0.26
	90	0.00	0.00



$I_{diffuse} = k_d * I * Cos(90*PI/180)$

Dot Product

$$\vec{A} \cdot \vec{B} = |\vec{A}|\,|\vec{B}|\cos\theta$$

the **dot product** is an algebraic operation that takes two equal-length sequences of numbers, and returns a **single number**.

https://en.wikipedia.org/wiki/Dot_product

$$I = I_p k_d |N||V_L| cos(\theta)$$

$$|N| = 1$$

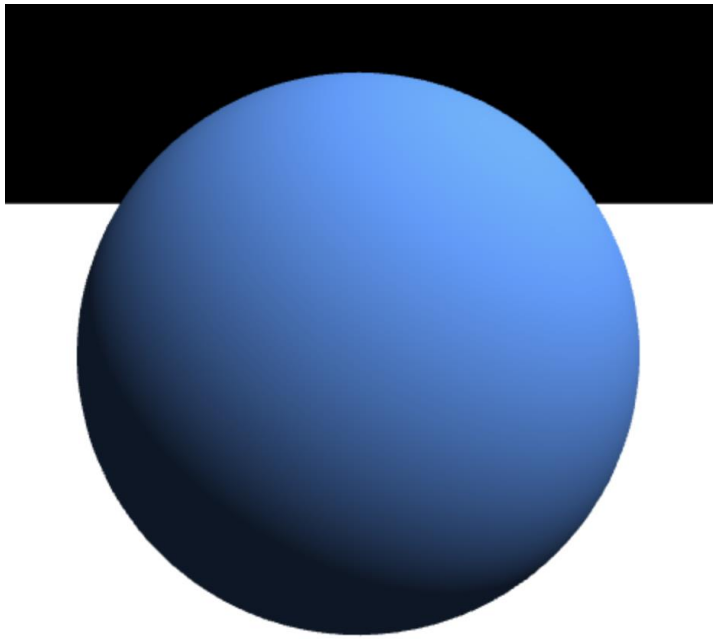
$$|L| = 1$$

$$N \cdot L = 1 \times 1 \times \cos\theta$$

$$\boxed{N \cdot L = \cos\theta}$$

$$I_{\text{diffuse}} = k_d I_{\text{light}} \max(0, \vec{N} \cdot \vec{L})$$

Lambert Shader



```
const frag = `
  precision mediump float;
  uniform vec3 uLightDir;
  varying vec3 vNormal;
  void main()
  {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(uLightDir);

    // Lambert diffuse
    float diffuse = max(dot(N, L), 0.0);

    vec3 baseColor = vec3(1.0, 1.0, 0.5);

    vec3 color = baseColor * (diffuse);

    gl_FragColor = vec4(color, 1.0);
    }
`;
```

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| \times |\mathbf{b}| \times \cos(\theta)$$

https://cs184.eecs.berkeley.edu/sp24/lecture/6-22/the-rasterization-pipeline

https://www.mathsisfun.com/algebra/vectors-dot-product.html

```
const uniforms =
{
	uTime: { value: 0.0 },
	uTexture: {
		value: new THREE.TextureLoader().load('https://threejs.org/examples/textures/uv_grid_opengl.jpg’)
	}
};
```

```
const fragmentShader = `
  precision mediump float;
  uniform float uTime;
  uniform sampler2D uTexture;
  varying vec2 vUv;

  void main() {
    //vec3 color = vec3(0.5, 1.0, 0.5);
    vec3 color = texture2D(uTexture, vUv).xyz;

    gl_FragColor = vec4(color, 1.0);
  }
`;
```

Ambient light

```
void main() {
  //vec3 color = vec3(0.5, 1.0, 0.5);
  vec3 N = normalize(vNormal);
  vec3 L = normalize(uLightDir);

  float diffuse =  max(dot(N, L), 0.0);

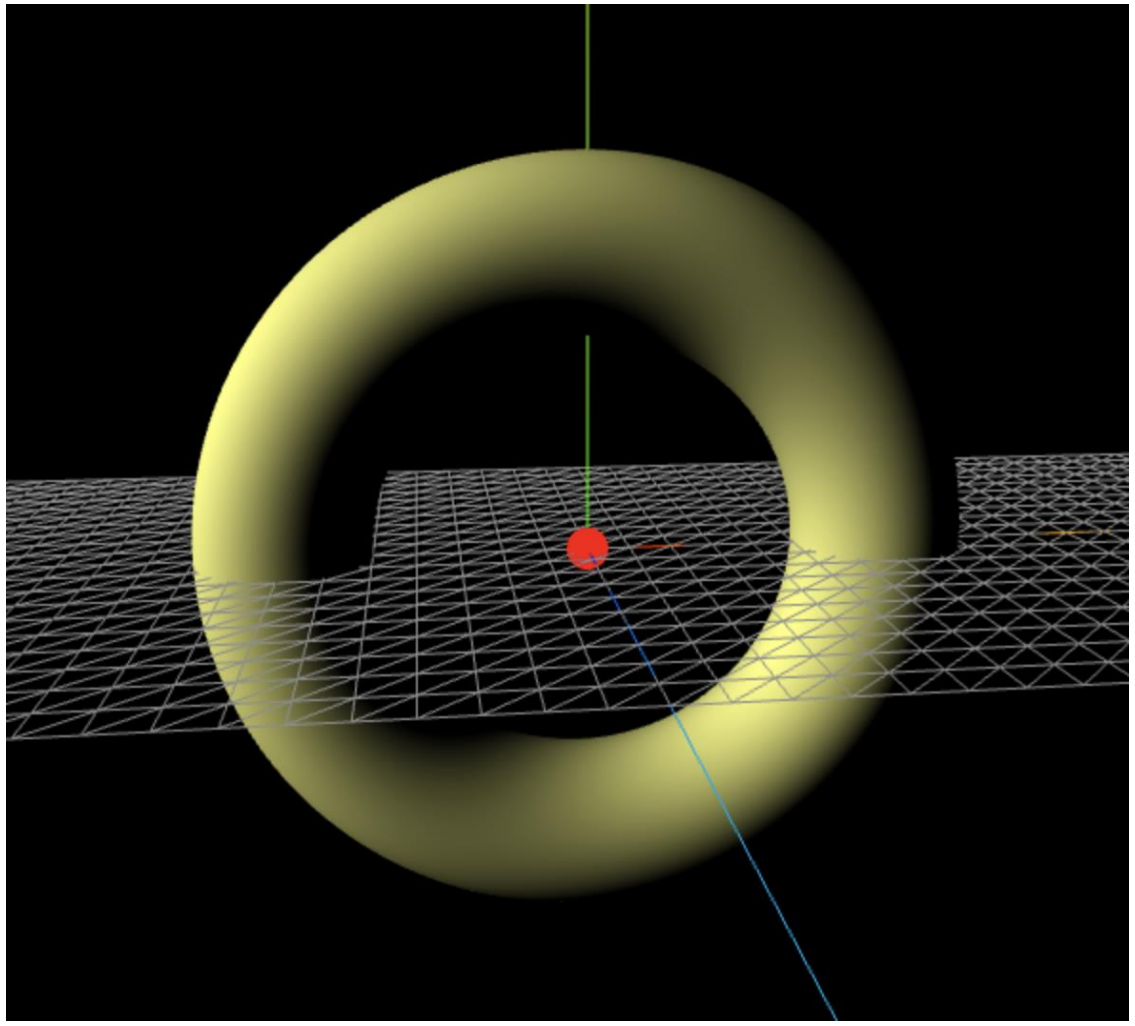
  float ambient = 0.3;

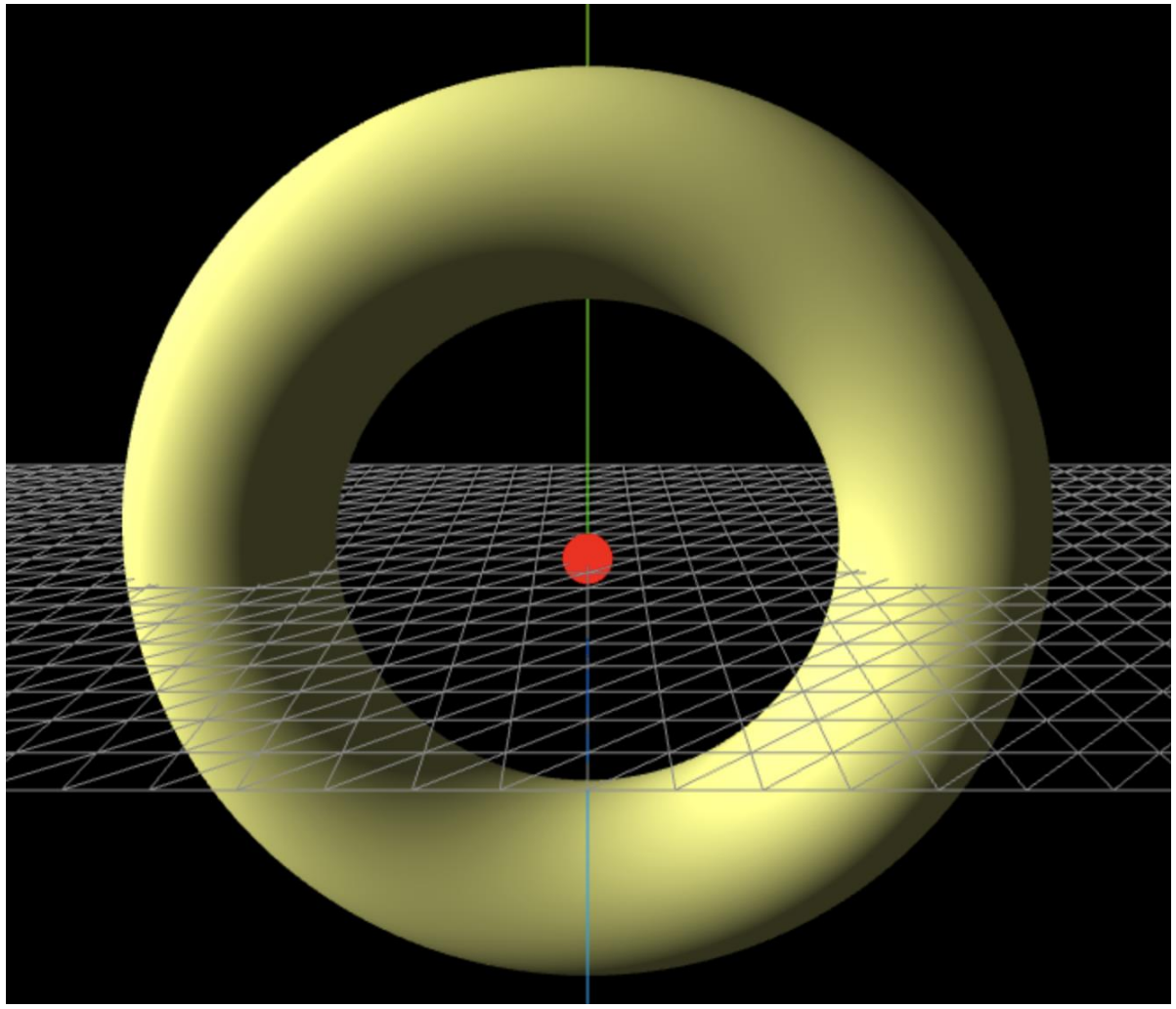

  vec3 baseColor = vec3(1.0, 1.0, 0.5);
  //vec3 baseColor = texture2D(uTexture, vUv).xyz;

  vec3 color = baseColor * (diffuse + ambient);
  gl_FragColor = vec4(color, 1.0);
}
```

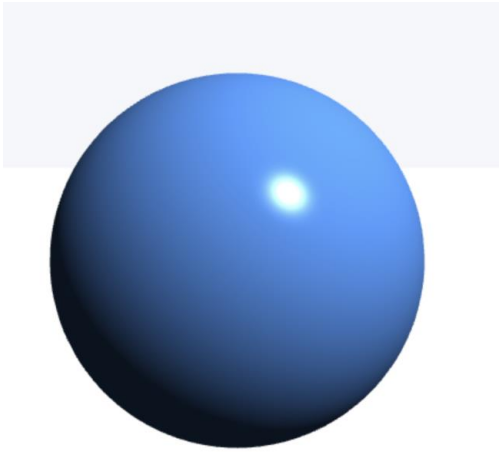
```
// Ambient
float ambient = 0.2;

vec3 color = baseColor * (diffuse + ambient);
```

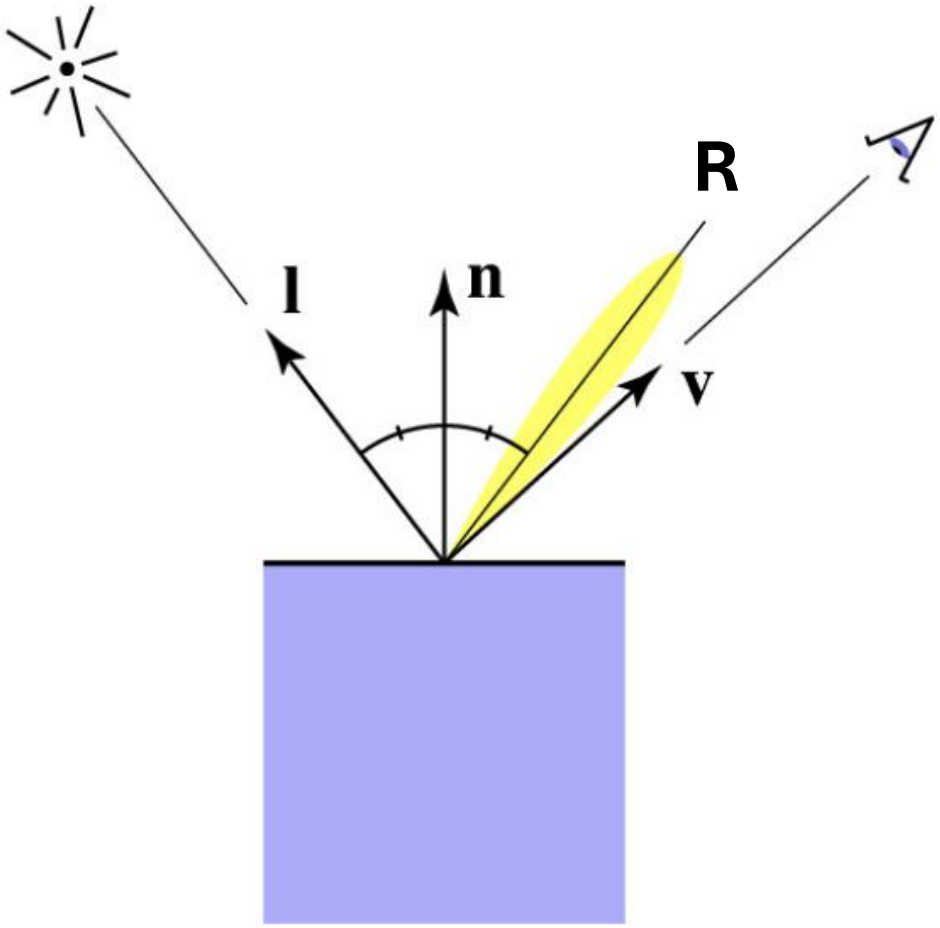




Phong Shader



$$I = k_s I_i (R * V)^p$$



https://medium.com/@leo6150631016/a-breif-note-about-phong-reflection-model-derivation-of-its-formula-different-approach-to-7711936cd231

Phong Shader

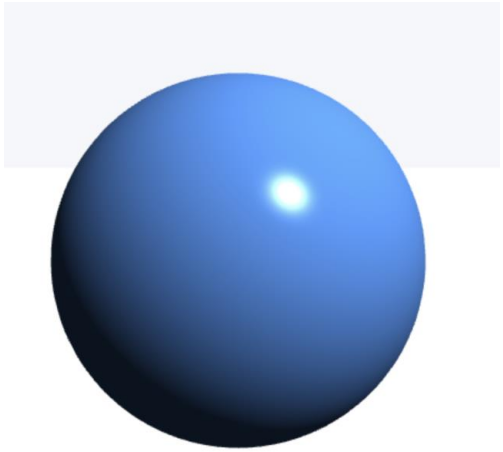
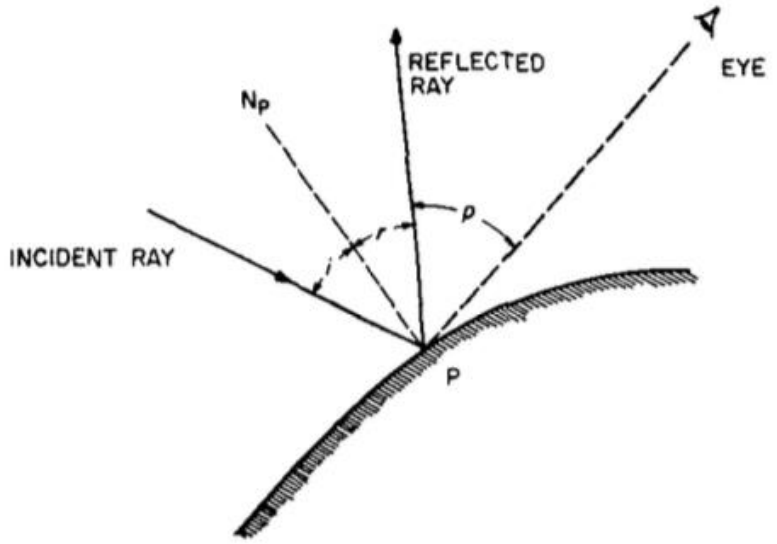
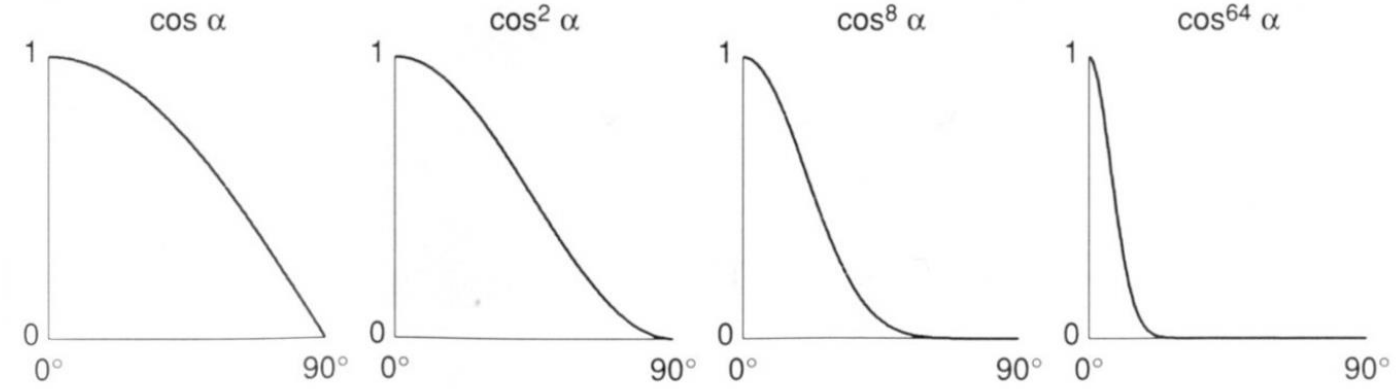
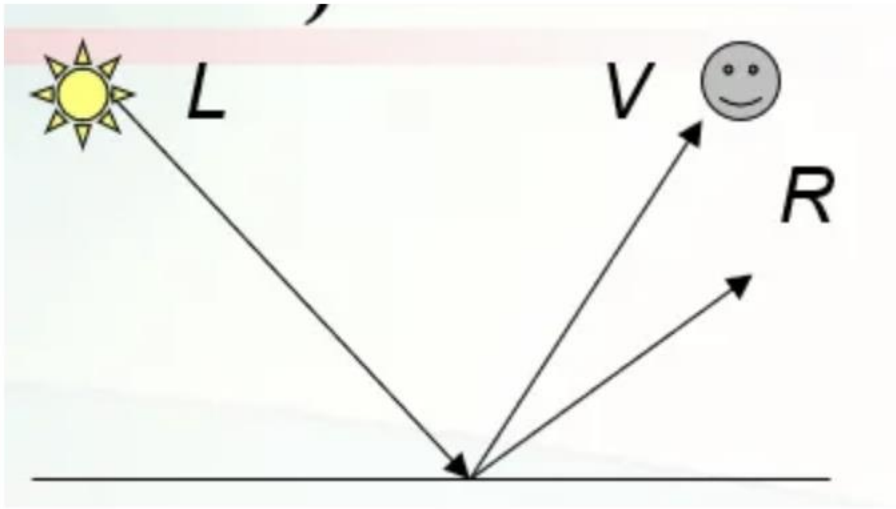


Fig. 6. Shading at a point.



Increasing p narrows the reflection lobe





https://medium.com/@leo6150631016/a-breif-note-about-phong-reflection-model-derivation-of-its-formula-different-approach-to-7711936cd231

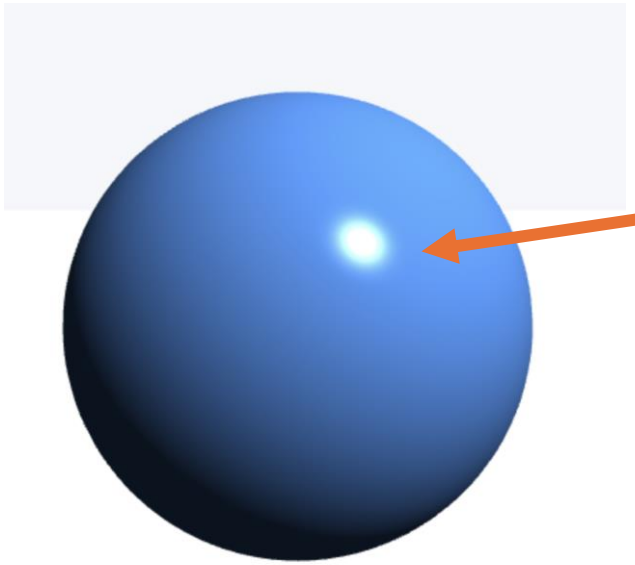
Phong Shader

vec3 reflect(vec3 I, vec3 N)

// return I - 2.0 * dot(N, I) * N

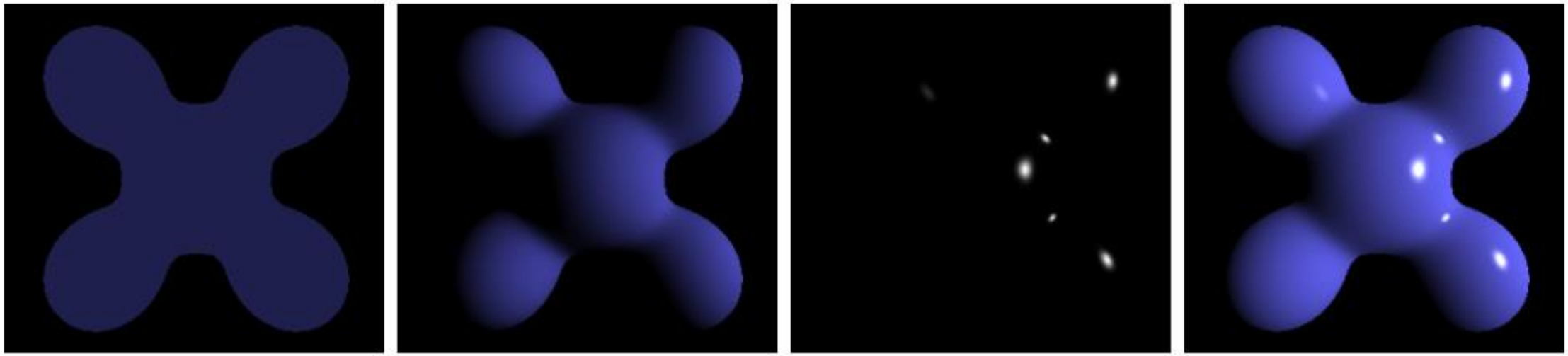
// I = เวกเตอร์ "ทิศทางแสงที่พุ่งชนผิว" (incident) ชี้เข้าหาผิว

// N = normal ของผิว (normalize)



```
const frag = `
  precision mediump float;
  varying vec3 vNormal;
  uniform vec3 uLightDir;
  varying vec3 vPosW;
  void main()
  {
    //lambert
    float diffuse = max(dot(vNormal, uLightDir),0.0);

    //phong
    vec3 R = reflect(-uLightDir, vNormal);
    vec3 V = normalize(cameraPosition - vPosW);
    float specular = max( pow( dot(R,V), 32.0 ) , 0.0);
    gl_FragColor = (vec4(1.0, 1.0, 0.5, 1.0) * diffuse) + vec4(specular);
  }
`;
```



https://en.wikipedia.org/wiki/Phong_reflection_model

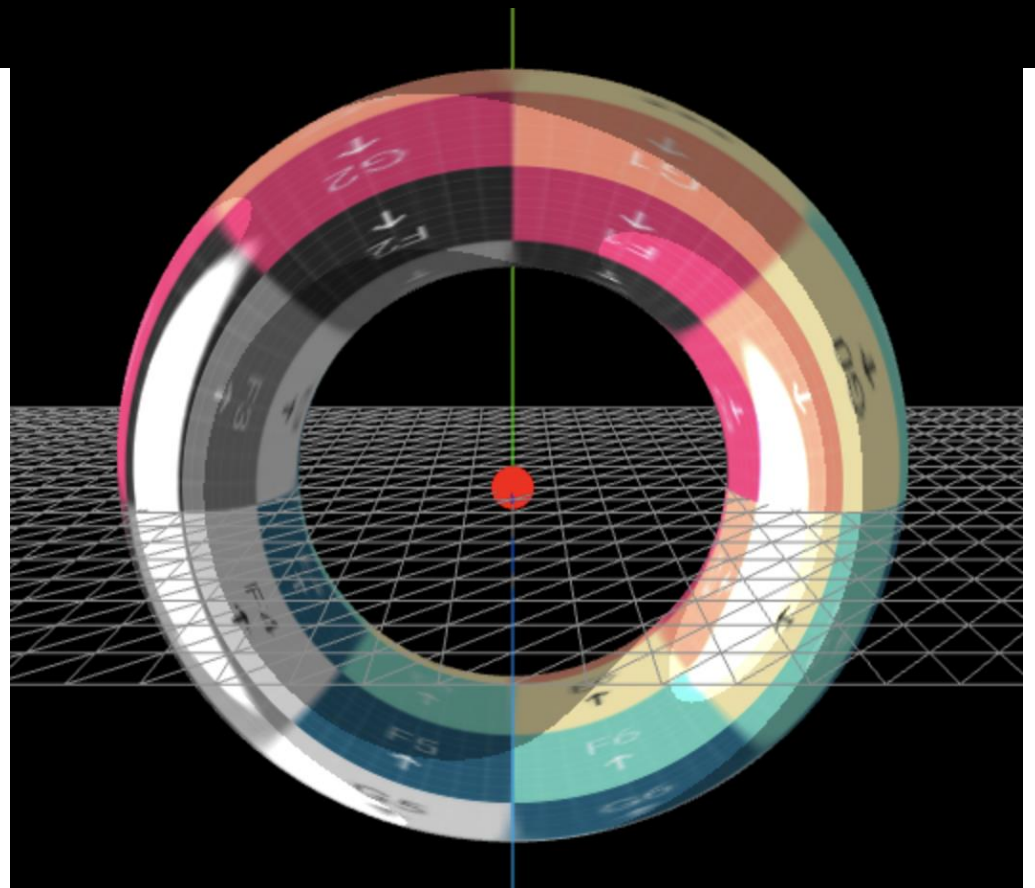
Function ที่ใช้งานบ่อย

ฟังก์ชัน	ความหมายย่อ	ตัวอย่าง GLSL	ใช้บ่อยกับ
max(a,b)	เลือกค่าสูงสุด	float d = max(dot(N,L), 0.0);	diffuse
min(a,b)	เลือกค่าต่ำสุด	float m = min(a, b);	ทั่วไป
clamp(x,lo,hi)	บีบค่าให้อยู่ในช่วง	float s = clamp(v, 0.0, 1.0);	saturate
pow(x,y)	ยกกำลัง	float spec = pow(max(dot(R,V),0.0), 32.0);	specular
abs(x)	ค่าสัมบูรณ์	float a = abs(sin(t));	ทั่วไป
sign(x)	คืนเครื่องหมาย	float sg = sign(x);	masking
floor(x)	ปัดลง	float f = floor(uv.x*5.0);	grid
ceil(x)	ปัดขึ้น	float c = ceil(uv.x*5.0);	grid
fract(x)	ส่วนเศษ	float g = fract(uv.x*5.0);	tiling
mod(x,y)	เลขเศษจากหาร	vec2 uv2 = mod(uv*4.0, 1.0);	Tiling
step(t,value)	แบ่งค่าเป็น 0, 1	specular = step(0.5, specular);	
smoothstep(tl,th,value)	แบ่งค่า เป็น 0.1 โดยมี smooth	specular = smoothstep(0.4, 0.6, specular);	


Ah? Take a look at that!
Test version. Not indicative of final game
UID: 111999

Cel Shading

```
const uniforms = {
        uTime: { value: 0.0 },
        uLightDir: { value: new THREE.Vector3(0.4, 1.0, 0.8).normalize() },
        uTex: { value: new
        THREE.TextureLoader().load('https://upload.wikimedia.org/wikipedia/commons/thumb/f/fd/UV_checker_Map
        _byValle.jpg/1200px-UV_checker_Map_byValle.jpg') }
};
```



```
float diffuse = max(dot(vNormal, uLightDir), 0.0);
float levels = 3.0;
diffuse = floor(diffuse * levels) / levels;
diffuse += 0.6;



vec3 R = reflect(-uLightDir, vNormal);
vec3 V = normalize(cameraPosition - vPosW);
float specular = clamp( pow( dot(R,V), 4.0 ) , 0.0, 1.0);


//specular = step(0.5, specular);
specular = smoothstep(0.4, 0.6, specular);
if(diffuse <= 0.6) specular = 0.0;


vec3 base = texture2D(uTex, vUv).rgb;
vec3 color = (base * diffuse) + vec3(specular);


gl_FragColor = vec4(color, 1.0);
```

Matrix สำคัญใน Vertex Shader

```
gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
```

Matrix	Data Type	Description
modelMatrix	mat4	Transform จาก Object space เป็น World space
viewMatrix	mat4	Transform จาก World space เป็น View space
projectionMatrix	mat4	Transform จาก View space เป็น Clip space
modelViewMatrix	mat4	viewMatrix * modelMatrix (Obj space เป็น View) (ใช้บ่อยจึงคูณไว้ให้เลย)
normalMatrix	mat3	Transform จาก normal เป็น View space
cameraPosition	vec3	พิกัดกล้อง

```
const outlineVertexShader = `
    uniform float uTime;

    varying vec2 vUv;
    varying vec3 vNormal;
    void main() {
    vUv = uv;
    vNormal = normal;
    vec3 pos = position;

    // เริ่มเขียน vertex shader ที่นี่
    pos.z += sin(pos.x * 5.0 + uTime) * 0.1;

    //scale up for outline
    pos += normal * 0.03;

    gl_Position = projectionMatrix * viewMatrix *
    modelMatrix * vec4(pos, 1.0);
    }
`;
```

```
const outlineFragmentShader = `
precision mediump float;


uniform float uTime;
uniform sampler2D uTexture;
varying vec2 vUv;
varying vec3 vNormal;
uniform vec3 uLightDir;


void main() {
gl_FragColor = vec4(0.1, 0.1, 0.1, 1.0);
}
`;
```

```
const outlineMaterial = new THREE.ShaderMaterial({
    vertexShader: outlineVertexShader,
    fragmentShader: outlineFragmentShader,
    uniforms,
    side: THREE.BackSide
});
```



```
// ใช้ segment หลายช่อง เพื่อให้ vertex shader ดัดรูปทรงได้ละเอียด
const geometry = new THREE.TorusKnotBufferGeometry(0.52, 0.2, 96, 16)
const material = new THREE.ShaderMaterial({
  vertexShader,
  fragmentShader,
  uniforms,
  side: THREE.DoubleSide
});

const outlineMaterial = new THREE.ShaderMaterial({
  vertexShader: outlineVertexShader,
  fragmentShader: outlineFragmentShader,
  uniforms,
  side: THREE.BackSide
});

const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);
mesh.position.set(1, 1, 0);

const outlineMesh = new THREE.Mesh(geometry, outlineMaterial);
scene.add(outlineMesh);
outlineMesh.position.set(1, 1, 0);
```